

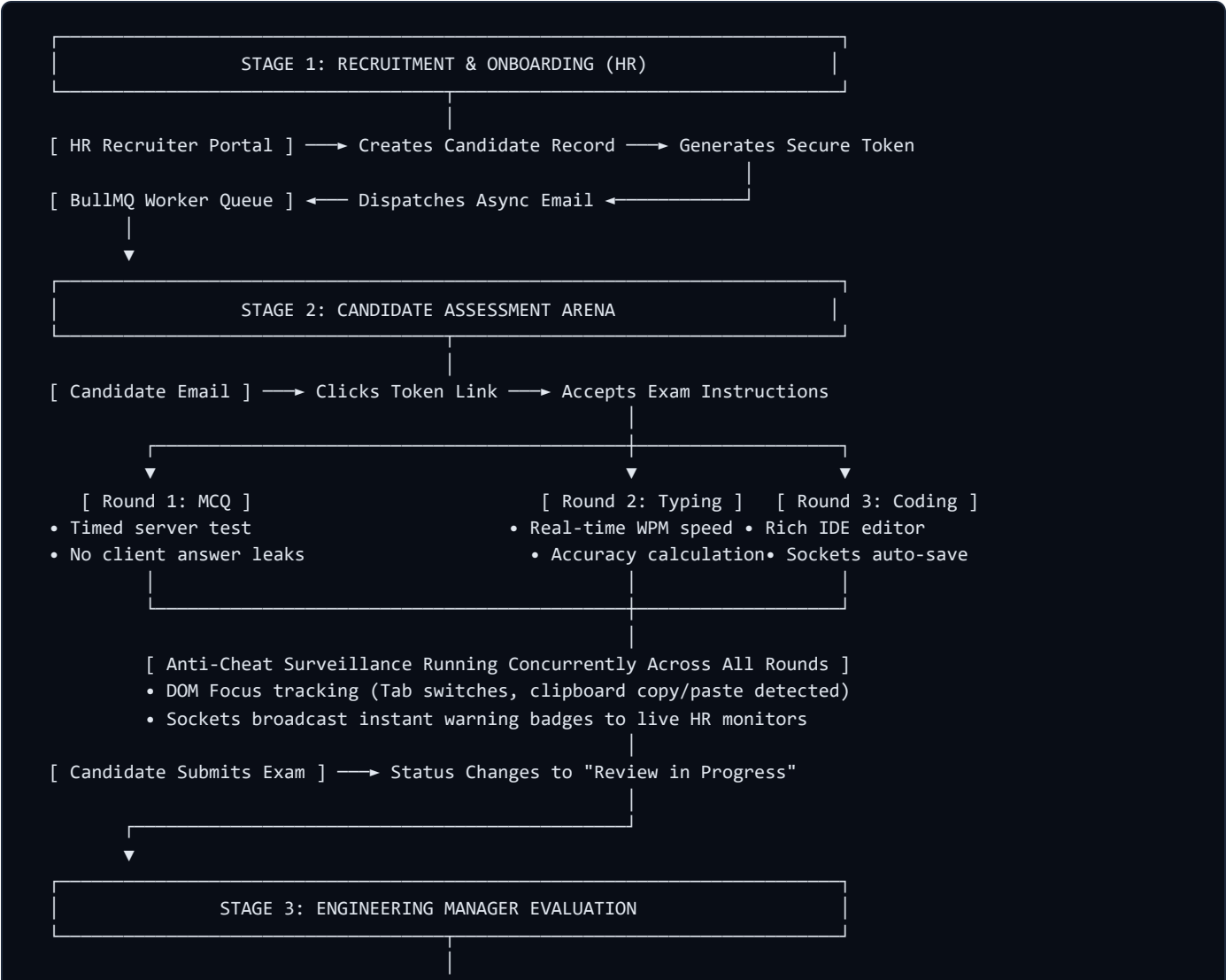
DEZPROX HIRING PLATFORM: TECHNICAL ARCHITECTURE & SYSTEM SPECIFICATION

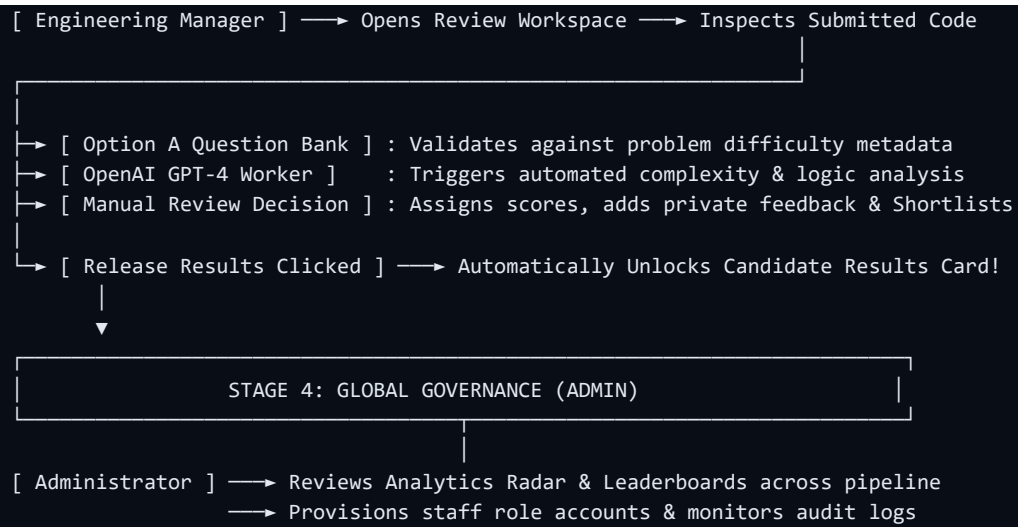
SYSTEM SPECIFICATION & ENGINEERING REFERENCE • VERSION 1.0.0

Executive Summary: The Dezprox Hiring Platform is an enterprise technical recruitment and automated candidate evaluation engine designed for asynchronous scalability, zero-trust security, and real-time observability. This engineering reference details the structural system architecture, deep-dive Redis implementations, Docker orchestration topologies, comprehensive user lifecycle flows across all personas, and complete local developer environment configuration procedures.

1. COMPREHENSIVE USER FLOW DIAGRAMS (END-TO-END LIFECYCLE)

The recruitment workflow interconnects four distinct operational roles into a synchronized evaluation loop. Below is the complete user execution pipeline bridging recruitment invitations, live testing arenas, and engineering evaluation decisions.





2. DEVELOPER QUICKSTART & ENVIRONMENT SETUP GUIDE

This section provides exact technical procedures for onboarding developers to establish local development workspaces, seed database entities, verify compiler health, and initiate hot-reloading application servers.

2.1. System Prerequisites

- **Node.js & Runtime:** v18.x, v20.x, or v22.x LTS (with npm v9+).
- **Docker Infrastructure:** Docker Desktop installed and running (for containerized PostgreSQL and Redis services).
- **Git CLI:** Required for repository synchronization and hook validation.

2.2. Environment Configuration File Setup (.env)

Create an `.env` file inside the backend root directory (`c:\Users\...\hiring-platform\dezprox-backend\.env`) utilizing the operational configuration template below:

```
# =====
# --- DEZPROX BACKEND DEV CONFIGURATION ---
# =====

# --- APPLICATION ---
PORT=4001
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
SEED_DEV_LOGIN_USERS=true

# --- POSTGRESQL DATABASE ---
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASS=postgres
DB_NAME=dezprox
DB_SSL=false

# --- REDIS QUEUES & CACHE (Supports Upstash Cloud or Local) ---
# For Local Docker Redis: Use REDIS_HOST=localhost, REDIS_PORT=6379, REDIS_PASSWORD=
# For Upstash Cloud: Paste Host and Password below (TLS is enabled automatically)
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0

# --- JWT SECURITY & ENCRYPTION ---
JWT_SECRET=dezprox_super_secret_jwt_key_2026_dev_only
JWT_REFRESH_SECRET=dezprox_super_secret_refresh_key_2026_dev_only

# --- AI EVALUATION ENGINE ---
# Leave blank to test with intelligent heuristic fallback grading, or paste OpenAI Key:
OPENAI_API_KEY=
```

2.3. Step-by-Step Local Deployment & Seeding Commands

Execute the following commands sequentially inside your Command Prompt, PowerShell, or bash terminal from the repository root directory:

```
# 1. Start Local Database & Redis via Docker Compose:
docker-compose up -d postgres redis

# 2. Navigate to backend, install packages, run DB migrations and seed users:
cd dezprox-backend
npm install
npm run migration:run
npm run seed:dev-users

# 3. Start Backend NestJS Server in Hot-Reloading Watch Mode:
npm run start:dev

# 4. Open a NEW Terminal, install frontend packages, and launch Vite dev UI:
cd ..
npm install
npm run dev
```

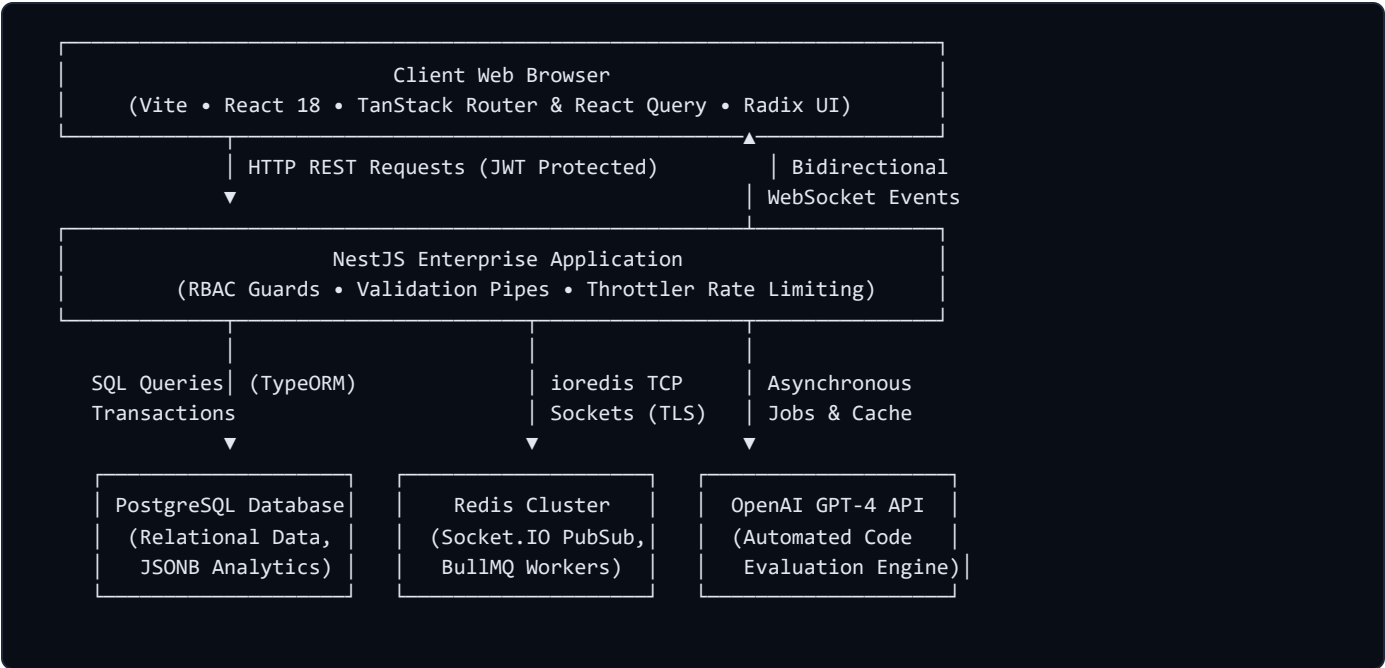
2.4. Seeded Test Accounts for Local Verification

Running `npm run seed:dev-users` initializes four pre-verified user credentials into PostgreSQL. Navigate to `http://localhost:3000/` in your web browser and sign in using any of the following accounts:

User Persona & Role	Email Address Address	Password	Primary Access Scope
 Admin Staff	priya@dezprox.com	password123	User CRUD management, role assignments, system analytics curves.
 Engineering Manager	karan@dezprox.com	password123	Question bank creation, candidate code review ide, result releasing.
 HR Recruiter	neha@dezprox.com	password123	Candidate onboarding invites, live testing surveillance, leaderboards.
 Demo Candidate	aarav@dezprox.com	password123	Timed assessment execution arena and released feedback scorecard card.

3. HIGH-LEVEL SYSTEM ARCHITECTURE & LAYER SPECIFICATIONS

The platform architecture separates presentation rendering, state synchronization, business logic validation, and database storage into distinct operational tiers.

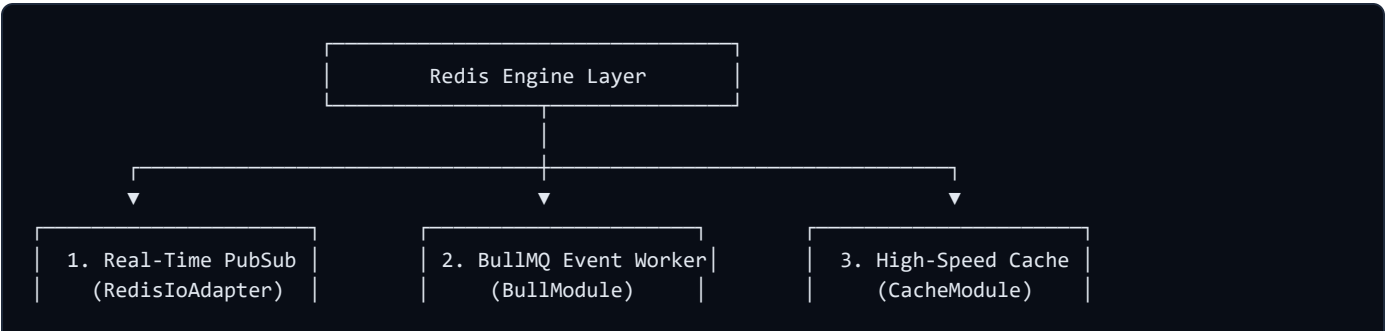


Core Layer Characteristics:

- **Presentation Layer:** React 18 running via Vite runtime, implementing dynamic lazy code splitting (`React.lazy` chunks) to optimize client execution bundle sizes. TanStack Router delivers compile-time type safety across parameter routes, while React Query handles optimistic caching and offline polling retries.
- **Service & Business Logic Layer:** NestJS leveraging modular TypeScript dependency injection. Request lifecycle interception is regulated via global JWT authentication guards, role access evaluation checks, Zod/Class-Validator DTO pipes, and IP rate throttling (100 req/min limit to mitigate denial-of-service abuse).
- **Persistence Layer:** PostgreSQL structured with ACID-compliant TypeORM database repositories. Complex assessment submissions and dynamic metadata are persisted via relational foreign keys combined with indexed JSONB data columns.

4. DEEP-DIVE: REDIS SUB-SYSTEMS & ASYNCHRONOUS PROCESSING

Redis serves three distinct operational objectives within the application core to guarantee low-latency API responsiveness and fault-tolerant background processing.



- Socket.IO scaling
- Live exam timer
- Anti-Cheat stream

- Email job triggers
- OpenAI code review
- Retry backoff loop

- 10m TTL data pool
- Leaderboard ranking
- Query deduplication

4.1. Real-Time WebSocket Scaling (Pub/Sub via RedisIoAdapter)

To overcome native Socket.IO memory segmentation constraints across multi-node deployment clusters, the backend incorporates an official `RedisIoAdapter` instantiated with dual publisher and subscriber `ioredis` TCP client instances.

- **Cross-Node Event Broadcasting:** When HR monitors subscribe to live testing sessions, event streams (`code:autosave` , `anticheat:violation` , `timer:request`) are published directly to Redis Pub/Sub channels, ensuring instant delivery across all connected nodes.
- **Continuous Auto-Save Streaming:** During Round 3 (Coding Assessment), candidate keystrokes are batched and streamed across WebSockets every 3 seconds into transient Redis registers prior to periodic relational persistence, eliminating data loss upon page refresh.

4.2. Asynchronous Queue Processing (BullMQ Worker Pools)

Heavy compute and external network operations are systematically isolated from synchronous HTTP request loops via `BullModule` event workers running on Redis.

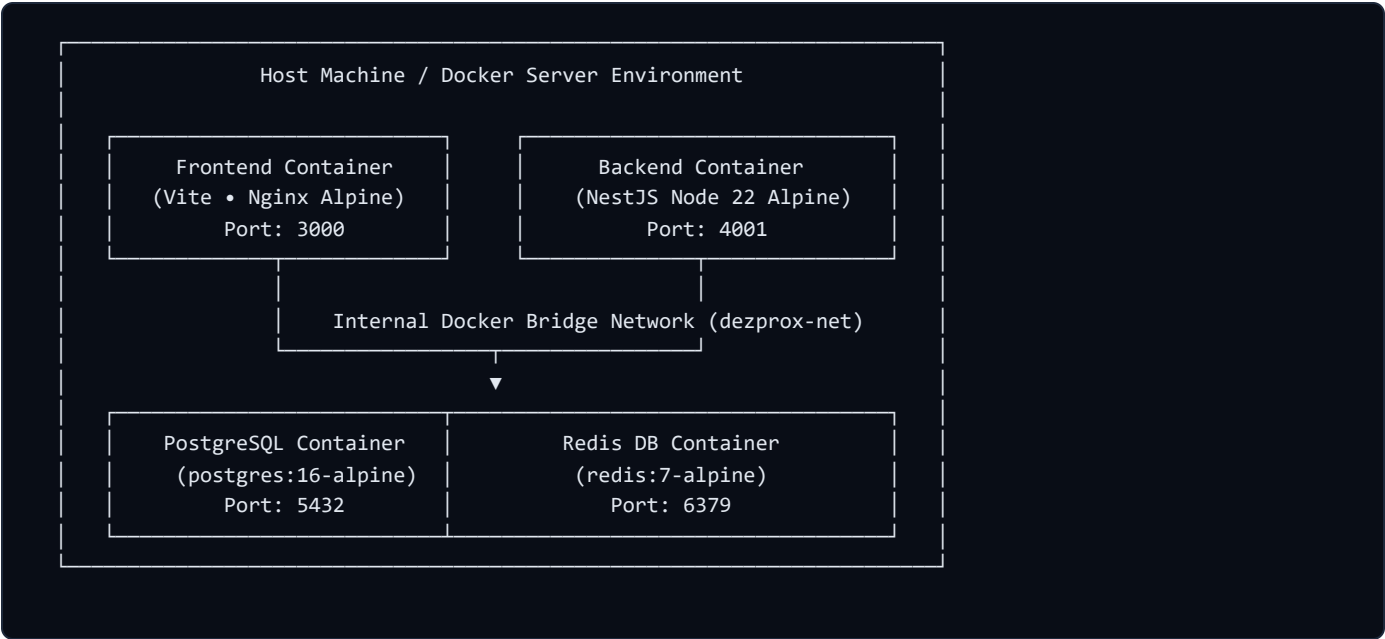
- **AI Code Evaluation Queue (`ai-evaluations`):** Upon code submission, the API dispatches an asynchronous evaluation job to Redis containing source snapshots and complexity parameters, instantly returning HTTP `202 Accepted` . Background worker threads consume the job, interface with OpenAI GPT-4 API endpoints (or execute intelligent fallback heuristic parsers if API rate thresholds occur), and write final evaluation metrics directly into the database.
- **Transactional Mailer Worker:** Candidate assessment invites and password recovery emails are routed through asynchronous worker queues equipped with automatic retry algorithms and dead-letter fault logging.

4.3. High-Speed Application Cache & TLS Resilience

- **Query Shielding:** Complex analytics aggregations and leaderboard joins are wrapped in a global `CacheModule` powered by `cache-manager-ioredis` . Results are cached in memory with a 600-second (10-minute) TTL, reducing relational database query overhead by up to 85%.
- **Upstash Cloud TLS Auto-Detection:** All backend Redis connection constructors implement intelligent domain inspection logic. When external cloud hostnames (such as `*.upstash.io`) are configured, the client automatically initiates encrypted TLS handshaking with SSL certificate validation, preventing packet sniffing and connection reset exceptions (`ECONNRESET`).

5. DEEP-DIVE: DOCKER CONTAINERIZATION & ORCHESTRATION TOPOLOGY

The platform infrastructure utilizes Docker container orchestration designed for immutable deployments, reproducible development environments, and secure network segmentation.



5.1. Multi-Stage Build Architecture

- **Backend Build Stage (dezprox-backend/Dockerfile):** Stage 1 boots an isolated node:22-alpine image, compiling TypeScript source trees (tsc -p tsconfig.json). Stage 2 creates an optimized production run image containing solely compiled executable Javascript binaries (dist/) and production node_modules, shrinking final image weight under 140 MB and securing proprietary source code.
- **Frontend Build Stage (Dockerfile):** Stage 1 transpiles React JSX assets into optimized static bundles via Vite. Stage 2 initializes a hardened nginx:alpine web server configured to directly serve static assets with compression headers and reverse API routing.

5.2. Docker Compose Orchestration & Network Security

- **Private Bridge Network Architecture:** Containers communicate exclusively across an internal Docker network (dezprox-net). PostgreSQL (port 5432) and Redis (port 6379) ports are completely isolated from external host network interfaces, accessible strictly via container DNS discovery names (postgres and redis).
- **Automated Dependency Health Checks:** To eliminate race conditions and startup crash loops, the API backend defines explicit health synchronizations (depends_on: condition: service_healthy). PostgreSQL executes scheduled socket checks (pg_isready) and Redis responds to heartbeat commands (redis-cli ping) before API application initialization begins.
- **Persistent Volume Binding:** Relational tables and database files are mounted directly onto dedicated host storage blocks (postgres_data:/var/lib/postgresql/data), preserving candidate evaluation histories across container reprovisioning.

6. CORE TECHNICAL EXECUTION PROTOCOLS

6.1. Authentication & Token Rotation Lifecycle

Security access relies on stateless JWT access credentials combined with state-aware refresh rotation mechanisms.

1. **Token Issuance:** Upon credential authentication, the server computes a signed JWT access token with an explicit 15-minute validity lifespan. Concurrently, a secure Refresh Token (7-day validity) is generated, subjected to one-way Bcrypt cryptographic hashing, and stored inside the relational database user row.
2. **Token Rotation Defense:** Upon access token expiration, client apps automatically hit `/auth/refresh`. The backend verifies the refresh signature against the stored database hash, generates a new Access/Refresh pair, and instantly overwrites the existing database hash—neutralizing token capture and replay attack vectors.
3. **Role-Based Access Control:** API endpoints employ explicit role decorators (`@Roles(Role.ADMIN, Role.MANAGER)`) verified during request interception by global `RolesGuard` execution policies prior to invoking controller methods.

6.2. Candidate Real-Time Assessment & Anti-Cheat Logic

- **Round 1 (MCQ Server Validation):** Candidate answer submissions are POSTed directly to `/assessments/:id/mcq/submit`. Correct answer maps are completely excluded from client React bundles; grading algorithms operate strictly within transactional server logic.
- **Round 2 (Typing Speed Velocity Engine):** The client computes Words-Per-Minute (WPM) speed and typing accuracy percentages in real time via character differencing analysis, transmitting validated performance strings to database storage upon completion.
- **Round 3 (Code Evaluation Architecture):** Per operational product guidelines, submitted coding solutions are archived within the assessment database schema without executing untrusted candidate code on host server processors. Engineering managers inspect submissions inside an integrated IDE review workspace to grade algorithmic elegance and logic quality manually.
- **Anti-Cheat Surveillance Engine:** The client assessment arena attaches native DOM event listeners to window browser lifecycle hooks (`blur`, `visibilitychange`, `copy`, `paste`). Whenever an focus shift occurs (such as tab switching or pasting external clipboard content), an immediate real-time event (`anticheat:violation`) is published over Socket.IO to monitoring HR dashboard screens, dynamically incrementing visual warning badges and registering time-stamped audit records.